

2.1 Overview of Clustering

Clustering is one of the most popular exploratory data analysis techniques which is used to get an deeper insights about the data. It can be defined as the task of classifying data in to subgroups where data points in the same subgroup (cluster) are very similar and data points in other clusters are different. In Big data analytics, clustering plays an important role in classifying different objects intended for finding the hidden structure in a data. Consider a situation wherein you need to analyze a set of data objects, and unlike classification the class label of each object is unknown. This condition occurs mainly in the case of large databases, where the process of defining class labels to a large number of objects is costly.

Clustering is the process of collecting and grouping similar data into classes or clusters. In other words, clustering is a process in which similar data is grouped into classes or clusters so that the objects within the same cluster or class have high similarity with respect to the dissimilar objects in another cluster or group. Often, the dissimilarities are evaluated on the basis of the attributes that describe an object. These attributes are also known as distance measures. The clustering is an unsupervised learning technique to group the similar objects. It is often used for exploratory analysis of data where predictions can't be made. It is specifically used to find the similarity between the objects based on their attributes. The similar objects are placed into a group called cluster. Each object is called data point where data point's lies in same cluster are similar to each other while points from different clusters are dissimilar in nature. In next section, we are going to study one of the most popular clustering algorithms called "K-means Clustering".

2.2 K-means Clustering

K-means clustering is one of the simplest and popular unsupervised machine learning algorithms. The K-means algorithm tries to partition the dataset into K pre-defined distinct non-overlapping subgroups (clusters) in iterative manner where each data point belongs to only one group.

Initially define a target number K, which refers to the number of centroids you need in the dataset. A centroid is the imaginary or real location representing the center of the cluster. The K-means algorithm identifies K number of centroids, and then allocates every data point to the nearest cluster, while keeping the centroids as small as possible.

To process the learning data, the K-means algorithm in data mining starts with a first group of randomly selected centroids, which are used as the beginning points for every

cluster, and then performs iterative (repetitive) calculations to optimize the positions of the centroids. It halts after creating and optimizing clusters when either the centroids have stabilized i.e. there is no change in their values because the clustering has been successful or the defined number of iterations has been achieved. This concept is represented in terms of algorithm as below.

Algorithmic Steps for K-Means Clustering

Step 1 : Let $X = \{X_1, X_2, \dots, X_n\}$ be the set of data points.

Step 2 : Arbitrarily select 'K' cluster centers denoted as $C_1, C_2, \dots, C_k$.

Step 3 : Calculate the distance between each data point with the cluster centers by using any distance measurers.

Step 4 : Assign the data points to the cluster center whose distance from the cluster center is minimum with respect to other cluster centers.

Step 5 : Recalculate the distance between each data point and its centers to get new cluster centers using mean.

Step 6 : Repeat from step 3 till there is no change in the cluster center

The pictorial representation of K-means algorithm is shown in Fig. 2.2.1.

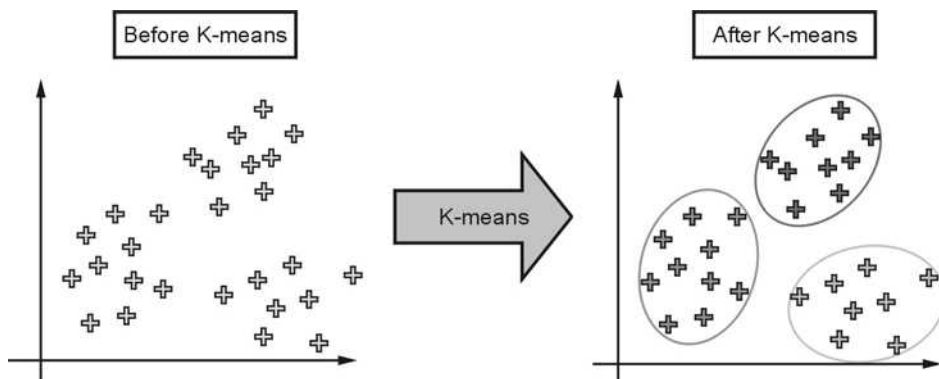


Fig. 2.2.1 Pictorial representation of K-means algorithm

2.2.1 One Dimensional Problem for K-means Clustering

Let, $X = \{2, 3, 4, 10, 11, 12, 20, 25, 30\}$ be the set of data points. Arbitrarily select $K = 2$ cluster centers $C_1 = 4$ and $C_2 = 12$.

Calculate the distance between each data point from X with the cluster centers $C_1 = 4$ and $C_2 = 12$ by using Euclidean distance.

The Euclidean distance can be calculated by formula

$$\begin{aligned} d(p,q) &= d(q,p) \\ &= \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \dots + (q_n - p_n)^2} \\ &= \sqrt{\sum_{i=1}^n (q_i - p_i)^2} \end{aligned}$$

For this sum, $d(i,m) = \sqrt{(C_i - X_m)^2}$ for $i = 1, 2$. And $m = 1, 2, \dots, 9$.

So, for cluster 1 and 2 the distances are calculated as shown in Table 2.2.1.

Cluster Centers	Data Points								
	2	3	4	10	11	12	20	25	30
$C_1 = 4$	2	1	0	6	7	8	16	21	26
$C_2 = 12$	10	9	8	2	1	0	8	13	18

Table 2.2.1 : Distance of data points with cluster centers C_1 and C_2 for iteration 1

From the above Table 2.2.1, the data points are clustered in accordance with the minimum distance from the cluster centers. So, the cluster center $C_1=4$ has the data points {2,3,4} and the cluster centers $C_2=12$ has the data points {10,11,12,20,25,30}. As per Step 4 of algorithm, we have assigned the data points to the cluster center whose distance from the cluster center is minimum with respect to other cluster centers.

Now, Calculate the new cluster center for each data points using mean.

$$\text{Mean} = \frac{1}{n} (\sum_{i=1}^n X_i)$$

So, for data points {2,3,4}, mean = 3 which is the new cluster center $C_1 = 3$ while for data points {10,11,12,20,25,30}, mean = 18, which is the new cluster center $C_2 = 18$. Now we have to repeat the same steps till there is no change in the cluster center.

Cluster centers	Data points								
	2	3	4	10	11	12	20	25	30
$C_1 = 3$	1	0	1	7	8	9	17	22	27
$C_2 = 18$	16	15	14	8	7	6	2	7	12

Table 2.2.2 : Distance of data points with cluster centers C_1 and C_2 for iteration 2

As per the above Table 2.2.2, the cluster center $C_1=3$ clusters data points {2,3,4,10} while the cluster center $C_2=18$ clusters data points {11,12,20,25,30}.

Now, Calculate the new cluster center for each data points using mean.

So, for data points {2,3,4,10}, mean = 4.75 which is the new cluster center $C_1 = 4.75$ while for data points {11,12,20,25,30}, mean = 18, which is the new cluster center $C_2 = 16.33$. Now we have to repeat the same steps till there is no change in the cluster center.

Cluster Centers	Data Points								
	2	3	4	10	11	12	20	25	30
C1 = 4.75	2.75	1.75	0.75	5.25	6.25	7.25	15.25	20.25	25.25
C2 = 16.33	14.33	13.33	12.33	6.33	5.33	4.33	3.67	8.67	13.67

Table 2.2.3 : Distance of data points with cluster centers C_1 and C_2 for iteration 3

Since we get the same data points for both the clusters as per the Table 2.2.3, so we can say that the final Cluster centers are $C_1 = 4.75$ and $C_2 = 16.33$.

2.3 Use Cases of K-means Clustering

K-means clustering has wide range of applications. Some of the popular use cases are listed below.

1. Document clustering
2. Delivery store optimization
3. Identifying crime localities
4. Customer segmentation
5. Fantasy league stat analysis
6. Fraud detection in insurance
7. Rideshare data analysis
8. Cyber-profiling criminals
9. Call record detail analysis
10. Automatic clustering of IT alerts

From above listed use cases, four popular use cases are explained below

1. Document clustering :

Document clustering refers to unsupervised classification (categorization) of documents into groups (clusters) in such a way that the documents in a cluster are similar, whereas documents in different clusters are dissimilar. The documents may be web pages, blog posts, news articles, or other text files.

2. Identifying crime localities :

Crime analysis and prevention is a systematic approach for identifying and analyzing patterns and trends in crime. Even though we cannot predict who all may be the victims of crime but can predict the place that has probability for its occurrence. K-means algorithm is done by partitioning data into groups based on their means. K-means algorithm has an extension called expectation - maximization algorithm where we partition the data based on their parameters. The following are the steps to demonstrate crime localities.

Step 1 : Create a new server on the web hosting sites available

Step 2 : Create two databases; one for storing the details of the authorized user and the other for storing details of the crime occurring in a particular location

Step 3 : The data can be added to the database using SQL queries

Step 4 : Create PHP scripts to add and retrieve data.

Step 5 : The PHP file to retrieve data converts the database in the JSON format.

Step 6 : This JSON data is parsed from the android so that it can be used.

Step 7 : The location added by the user from the android device is in the form of address which is converted in the form of latitudes and longitude that is further added to the online database.

Step 8 : The added locations are marked on the Google map.

Step 9 : The various crime types used are Robbery, Kidnapping, Murder, Burglary and Rape. Each crime type is denoted using a different color marker.

Step 10 : The crime data plotted on the maps is passed to the K - means algorithm.

Step 11 : The data set is divided into different clusters by computing the distance of the data from the centroid repeatedly.

Step 12 : A different colored circle is drawn for different clusters by taking the centroid of the clusters as the center where the color represents the frequency of the crime

Step 13 : This entire process of clustering is also performed on each of the crime types individually.

Step 14 : In the end, a red colored circle indicates the location where safety measures must be adopted.

From the clustered results it is easy to identify crime prone areas and can be used to design precaution methods for future. The classification of data is mainly used to distinguish types of preventive measures to be used for each crime. Different crimes require different treatment and it can be achieved easily using this application. The clustering technique is effective in terms of analysis speed, identifying common crime patterns and crime prone areas for future prediction. The developed application has promising value in the current complex crime scenario and can be used as an effective tool by Indian police and enforcement of law organizations for crime detection and prevention.

3. Cyber-profiling criminals :

The activities of Internet users are increasing from year to year and have had an impact on the behavior of the users themselves. Assessment of user behavior is often only based on interaction across the Internet without knowing any others activities. The log activity can be used as another way to study the behavior of the user. The Log Internet activity is one of the types of big data so that the use of data mining with K-Means technique can be used as a solution for the analysis of user behavior. This study is the process of clustering using K-Means algorithm which divides into three clusters, namely high, medium, and low. The cyber profiling is strongly influenced by environmental factors and daily activities. For investigation, the cyber-profiling process gives a good, contributing to the field of forensic computer science. Cyber Profiling is one of the efforts to know the alleged offenders through the analysis of data patterns that include aspects of technology, investigation, psychology, and sociology. Cyber Profiling process can be directed to the benefit of:

1. Identification of users of computers that have been used previously.
2. Mapping the subject of family, social life, work, or network-based organizations, including those for whom he/she worked.
3. Provision of information about the user regarding his ability, level of threat, and how vulnerable to threats to identify the suspected abuser

Criminal profiles generated in the form of data on personal traits, tendencies, habits, and geographic-demographic characteristics of the offender (for example: age, gender, socio-economic status, education, origin place of residence). Preparation of criminal profiling will relate to the analysis of physical evidence found at the crime scene, the process of extracting the understanding of the victim (victimology), looking for a modus

operandi (whether the crime scene planned or unplanned), and the process of tracing the perpetrators were deliberately left out (signature)

4. Fraud detection in Insurance :

Insurance industry is one of the most important issues in both economy and human being life in modern societies which awards peace and safety to the people by compensating the financial risk of detriments and losses. This industry, like others, requires choosing some strategies to obtain desired ranking and remain in competitive market. One of efficient factors which affects enormous decision makings in insurance is paying attention to important information of customers and bazar that each insurance company stores it in its own database. But with daily increasing data in databases, although hidden knowledge and pattern discovery using usual statistical methods is complicated, time-consuming and impossible to achieve. Data mining is a powerful approach for extracting hidden knowledge and patterns on massive data to guide insurance industry. For example, one of the greatest deleterious challenges here is interacting between insurance companies and policyholders which create a feasible situation for fraudulent claims. Due to importance of this issue, after investigating different ways of fraudulent crimes in insurance, we use K-Means clustering technique to find fraud patterns in automobile insurance include body and third-party. Our experimental results indicate a high accuracy when have been compared with statistical information extracted from data sets. Outcomes show significant relations among efficient factors in similar fraud cases.

2.4 Determining the Number of Clusters

One of the important issues of K-means is to find the appropriate number of clusters in a data set i.e. to specify the number of clusters “K” to be generated. The optimal number of clusters is somehow subjective and depends on the method used for measuring similarities and the parameters used for partitioning. These methods include direct methods and statistical testing methods :

1. Direct methods which consists of optimizing a criterion, such as the within cluster sums of squares or the average silhouette. The corresponding methods are named as elbow and silhouette methods, respectively.
2. Statistical testing methods which consists of comparing evidence against null hypothesis. An example is the gap statistic.

2.4.1 Elbow Method

The Elbow method looks at the total within-cluster sum of square (WSS) as a function of the number of clusters. One should choose a number of clusters so that adding another cluster doesn't improve much better the total WSS. The total WSS measures the compactness of the clustering and we want it to be as small as possible. WSS is the sum of squares used to determine the optimal value of K. The formula to find WSS is given as

$$WSS = \sum_{i=1}^m d(p_i, q^{(i)})^2$$

Where p_i represents data point and $q^{(i)}$ represents the cluster center

The optimal number of clusters can be defined as follows

1. Compute clustering algorithm for different values of K. For instance, by varying K from 1 to 10 clusters.
2. Calculate the Within-Cluster-Sum of Squared Errors for different values of K, and choose the K for which WSS becomes first starts to diminish. The Squared Error for each point is the square of the distance of the point from its cluster center.
 - The WSS score is the sum of these Squared Errors for all the points.
 - Any distance metric like the Euclidean Distance or the Manhattan Distance can be used.
3. Plot the curve of WSS according to the number of clusters K as shown in Fig. 2.4.1.
4. Location of a bend (knee) in the plot is generally considered as an indicator of the appropriate number of clusters.
5. In the plot of WSS-versus-K, this is visible as an elbow.

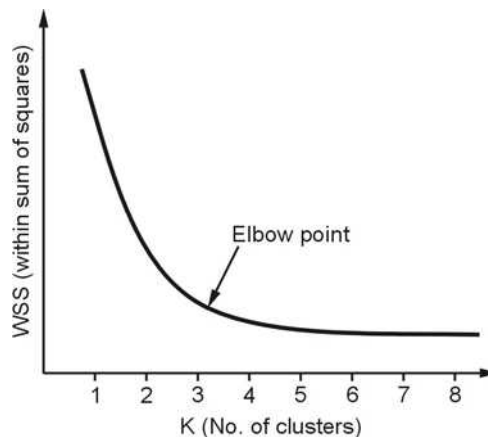


Fig. 2.4.1 Elbow method

From the above Fig. 2.4.1, we conclude that for the given number of clusters, the elbow point is found at $K = 3$. So for this problem, the maximum number of clusters would be 3.

2.4.2 Average Silhouette Method

The Silhouette method is used for interpretation and validation of consistency within clusters of data. It provides graphical representation of how well each object has been classified. It measures the quality of a clustering. That is, it determines how well each object lies within its cluster. A high average silhouette width indicates a good clustering. Average silhouette method computes the average silhouette of observations for different values of K . The optimal number of clusters K is the one that maximize the average silhouette over a range of possible values for K . The algorithm can be computed as follow :

1. Compute clustering algorithm (e.g., K-means clustering) for different values of K . For instance, by varying K from 1 to 10 clusters.
2. For each K , calculate the average silhouette of observations.
3. Plot the curve of according to the number of clusters K .
4. The location of the maximum is considered as the appropriate number of clusters.

The silhouette value measures the similarity of data point in its own cluster (cohesion) compared to o^{th} sample into two or more homogeneous sets (leaves) based on the most significant differentiators in your input variables. er clusters (separation).

The range of the Silhouette value is between +1 and – 1. A high value is desirable and indicates that the point is placed in the correct cluster. If many points have a negative Silhouette value, it may indicate that we have created too many or too few clusters.

The Silhouette Value $s(i)$ for each data point i is defined as follows :

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad \text{if } |C_i| > 1$$

and

$$s(i) = 0, \text{ if } |C_i| = 1$$

$s(i)$ is defined to be equal to zero if i is the only point in the cluster. This is to prevent the number of clusters from increasing significantly with many single-point clusters.

Here, $a(i)$ is the measure of similarity of the point i to its own cluster. It is measured as the average distance of i from other points in the cluster.

For each data point $i \in C_i$ (data point i in the duster C_i), let

$$a(i) = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \neq j} d(i,j)$$

Similarly, $b(i)$ is the measure of dissimilarity of i from points in other clusters.

For each data point $i \in C_i$, we now define

$$b(i) = \min_{i \neq j} \frac{1}{|C_j|} \sum_{j \in C_j} d(i, j)$$

$d(i, j)$ is the distance between points i and j . Generally, Euclidean Distance is used as the distance metric.

2.4.3 Gap Statistic Method

The gap statistic approach can be applied to any clustering method. The gap statistic compares the total within intra-cluster variation for different values of K with their expected values under null reference distribution of the data. The estimate of the optimal clusters will be value that maximizes the gap statistic. This means that the clustering structure is far away from the random uniform distribution of points.

2.5 Diagnostics

In WSS, the heuristic value must be chosen to get the desired output which can be provided at least several possible values of K . When numbers of attributes are relatively small, a common approach is required to further refinement of distinct identified clusters. The example of distinct clusters is shown in Fig. 2.5.1 (a).

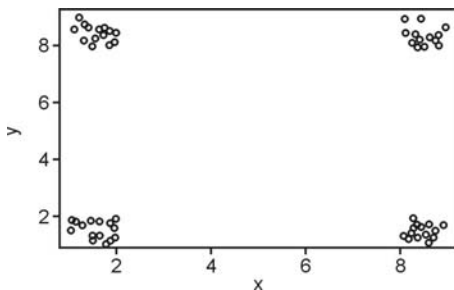


Fig. 2.5.1 (a) Example of distinct clusters

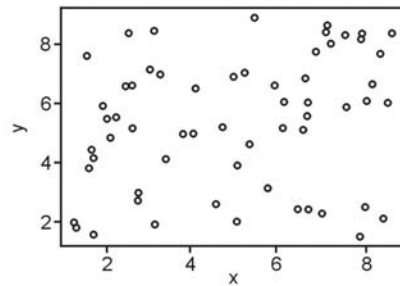


Fig. 2.5.1 (b) Example of less obvious clusters

To resolve the problem of distinct clusters, the above three questions needs to be considered.

Q 1) whether the clusters are separated from each other's?

Q 2) whether any cluster has only few points?

Q 3) whether any centroid appears to be too close to each other's?

The solution to above three questions might results in to less obvious clusters as shown in Fig. 2.5.1 (b).

2.6 Reasons to Choose and Cautions

As we know that, K-means is a simplest method for defining clusters, where clusters and their associated centroids are identified for easy assignment of new objects. Each new object has its own distance from the closest centroid. But, as this method is unsupervised, some of the consideration has to be look by practitioner like type of object attributes included in the analysis, unit of measure for each attribute, rescaling of attributes without disproportionate results and several additional considerations which are explained below.

The first consideration is analysis of object attribute, where it is important to understand what attributes will be known at the time a new object is assigned to a cluster rather than which object attributes are used in the analysis. Usually, Data Scientist may have a choice of a dozen or more attributes to use in the clustering analysis. But whenever possible they reduce the number of attributes in clustering analysis as too many attributes can minimize the impact of the most important variables. So, one must identify the highly correlated attributes and use only one or two of the correlated attributes in the clustering analysis.

The second consideration is Units of Measure, where each attribute must be defined with some or other units of measure (like gram or kilogram for weight and meters or centimeters for a patient's height). So, all the attributes in a given cluster must have same unit of measure. The dissimilar units of measure for a different attribute may result into inconsistency in results.

The third consideration is Rescaling; where attributes that are common in clustering analyses can have differ in magnitude from the other attributes. With the rescaled attributes, the borders of the resulting clusters fall somewhere between the two earlier clustering analyses. Some practitioners also subtract the means of the attributes to center the attributes around zero. However, this step is unnecessary because the distance formula is only sensitive to the scale of the attribute, not its location.

The additional consideration includes, defining the starting positions of the initial centroid as it is sensitive for working of K-means algorithm. Thus, it is important to rerun the k-means analysis several times for a particular value of K to ensure the cluster results provide the overall minimum WSS.

2.7 Classification

In machine learning and statistics, classification is a supervised learning approach in which the computer program learns from the data input given to it and then uses this

learning to classify new observation. Classification is technique to categorize our data into a desired and distinct number of classes where we can assign label to each class. Applications of classification are, speech recognition, handwriting recognition, biometric identification, document classification etc. A Binary classifiers classifies 2 distinct classes or with 2 possible outcomes while Multi-Class classifiers classifies more than two distinct classes. The different types of classification algorithms in Machine Learning are :

1. Linear Classifiers : Logistic Regression, Naive Bayes Classifier
2. Nearest Neighbor
3. Support Vector Machines
4. Decision Trees
5. Boosted Trees
6. Random Forest
7. Neural Networks

2.7.1 Overview of Decision Tree Classifier

Given a data of attributes together with its classes, a decision tree produces a sequence of rules that can be used to classify the data. Decision Tree, as its name says, makes decision with tree-like model. It splits the samples into one or more similar sets (leaves) based on the most significant differentiators in the input attributes. To choose a differentiator (predictor), the algorithm considers all features and does a binary split on them. It will then choose the one with the least cost (i.e. highest accuracy), and repeats recursively, until it successfully splits the data in all leaves (or reaches the maximum depth). The final result is a tree with decision nodes and leaf nodes where decision node has two or more branches and a leaf node represents a classification or decision. The uppermost decision node in a tree corresponds to the best predictor and is called the root node. Decision trees can handle both categorical and numerical data.

2.7.2 Terms used in Decision Trees

1. **Root node** : The root node represents the entire dataset which further gets divided into two or more standardized subsets.
2. **Splitting** : The Splitting is a process of dividing the nodes into two or more sub-nodes.
3. **Pruning** : The Pruning is the procedure of removing the sub-nodes of a decision node which is contradictory to process of splitting.

4. **Branch / sub-tree** : The branch or sub-tree are the sub part of an entire tree.
5. **Decision node** : The Decision node is generated when a sub-nodes are splitted into further sub-nodes.
6. **Parent and child node** : The node which is divided into sub-nodes is called parent node and sub-nodes are called child nodes.
7. **Leaf / terminal node** : The nodes which do not get further splitted are called as Leaf or Terminating node.

2.7.3 Advantages and Disadvantages of Decision Trees

The advantages of are listed below

- Decision trees generate understandable rules.
- Decision trees perform classification without requiring much computation.
- Decision trees are capable of handling both continuous and categorical variables.
- Decision trees provide a path to find out which fields are most important for prediction or classification.

The disadvantages of are listed below

- Decision trees are less appropriate for estimation tasks where the goal is to predict the value of a continuous attribute.
- Decision trees are prone to errors in classification problems with many classes and a relatively small number of training examples.
- Decision trees can be computationally expensive to train. The process of growing a decision tree is computationally expensive. At each node, each candidate splitting field must be sorted before its best split can be found. In some algorithms, combinations of fields are used and a search must be made for optimal combining weights. Pruning algorithms can also be expensive since many candidate sub-trees must be formed and compared.
- **Over fitting** : Decision-tree learners can create over-complex trees that do not generalize the data well. This is called over fitting.
- The over fitting is not appropriate for continuous variables : While working with continuous numerical variables, decision tree loses information, when it categorizes variables in dissimilar categories.
- Decision trees can be unbalanced because small discrepancies in the data might result in a completely different tree. This is called variance, which needs to be lowered by methods like bagging and boosting.

- Greedy algorithms cannot guarantee to return the globally optimal decision tree. This can be mitigated by training multiple trees, where the features and samples are randomly sampled with replacement.
- Decision tree learners create *biased* trees if some classes dominate. It is therefore recommended to balance the data set prior to fitting with the decision tree.
- Information gain in a decision tree with categorical variables gives a biased response for attributes with greater no. of categories.
- Generally, it gives low prediction accuracy for a dataset as compared to other machine learning algorithms.

Calculations can become complex when there are many class label.

Basically, there are three algorithms for creating a decision tree; namely Iterative Dichotomiser 3 (ID3), Classification And Regression Trees (CART) and C4.5. The next section 2.8 will describe the above three decision tree algorithms in detail.

2.8 Decision Tree Algorithms

In general, the objective of a decision tree algorithm is to construct a tree T from a training set S . The general algorithm constructs a sub trees like $T_1, T_2, \dots, T_n$ for the subsets of S recursively until one of the following criteria is met :

- All the leaf nodes in the tree satisfy the minimum purity threshold.
- The tree cannot be further split with the preset minimum purity threshold.
- Any other stopping criterion is satisfied (such as the maximum depth of the tree).

The first step in constructing a decision tree is to choose the most informative attribute i.e. Root node. A common way to identify the most informative attribute is to use entropy-based methods, which is used by decision tree learning algorithms such as ID3. The ID3 algorithm is explained in next section.

2.8.1 ID3 Algorithm

The ID3 algorithm makes use of entropy function and information gain. The ID3 method selects the most informative attribute based on two basic measures like Entropy, which measures the impurity of an attribute and Information gain, which measures the purity of an attribute.

Let us see an example how the decision tree is formed using ID3 algorithm. As shown in Table 2.8.1, there are five attributes like Outlook, Temperature, Humidity, Windy and

Play Cricket. Make a decision tree that predicts whether cricket will be played on the day.

Sr. No	Outlook	Temperature	Humidity	Windy	Play cricket
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	High	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Table 2.8.1 : Attributes for ID3 algorithms

The play cricket is the final outcome which depends on the other four attributes. To start with we have to choose the root node. To choose the best attribute we find entropy which specifies the uncertainty of data and information gain represented as,

$$\text{Entropy} = \frac{-p}{p+n} \log_2 \left(\frac{p}{p+n} \right) - \frac{n}{p+n} \log_2 \left(\frac{n}{p+n} \right)$$

Average information can be calculated as,

$$I(\text{Attribute}) = \sum \frac{p_i + n_i}{p+n} \text{Entropy}(A)$$

And the Information Gain can be calculated as :

$$\text{Information Gain} = \text{Entropy}(S) - I(\text{Attribute})$$

The ID3 algorithm to create the decision tree will work as follows

1. Calculate the Entropy for Data-Set Entropy(S)
2. For Every Attribute do the following

- a. Calculate the Entropy for all other Values Entropy(A)
 - b. Find Average Information Entropy for the Current attribute
 - c. Calculate Information Gain for the current attribute
3. Pick the Highest gain attribute
 4. Repeat the above steps until we get the desired decision tree

As seen from the Table 2.8.1, the attribute play cricket has Yes/Positive (p) = 9 and No/Negative (n) = 5. So using the formula we calculate the Data-Set Entropy.

$$\text{Entropy} = \frac{-9}{9+5} \log_2 \left(\frac{9}{9+5} \right) - \frac{5}{9+5} \log_2 \left(\frac{5}{9+5} \right) = 0.940$$

Next step is to calculate the entropy for each attribute. Let us start with outlook, which has three outcomes like sunny, rainy and overcast. As seen from the Table 2.8.2 we determine the entropy for each case.

Outlook	Play Cricket
Sunny	No
Sunny	No
Sunny	No
Sunny	Yes
Sunny	Yes

Outlook	Play Cricket
Rainy	Yes
Rainy	Yes
Rainy	No
Rainy	Yes
Rainy	No

Outlook	Play Cricket
Overcast	Yes
Overcast	Yes
Overcast	Yes
Overcast	Yes

Outlook	p	n	Entropy
Sunny	2	3	0.971
Rainy	3	2	0.971
Overcast	4	0	0

Table 2.8.2 : Relation of attribute outlook with play cricket

Now we calculate average information entropy for outlook

$$I(\text{Outlook}) = \frac{p(\text{sunny}) + n(\text{sunny})}{p + n} * \text{Entropy}(\text{outlook} = \text{sunny}) + \frac{p(\text{rainy}) + n(\text{rainy})}{p + n} *$$

$$\text{Entropy}(\text{outlook} = \text{rainy}) + \frac{p(\text{overcast}) + n(\text{overcast})}{p + n} * \text{Entropy}(\text{outlook} = \text{overcast})$$

$$\text{i.e. } I(\text{outlook}) = \frac{3+2}{9+5} * 0.971 + \frac{2+3}{9+5} * 0.971 + \frac{4+0}{9+5} * 0 = \mathbf{0.693}$$

Now we calculate information gain for the attribute outlook

$$\text{Gain} = \text{Entropy}(S) - I(\text{Outlook}) = 0.940 - 0.693 = \mathbf{0.247}$$

Now, repeat the same procedure for Temperature as shown Table 2.8.3.

Temperature	Play Cricket
Hot	No
Hot	No
Hot	No
Hot	Yes

Temperature	Play Cricket
Mild	Yes
Mild	No
Mild	Yes
Mild	Yes
Mild	Yes
Mild	No

Temperature	Play Cricket
Cool	Yes
Cool	No
Cool	Yes
Cool	Yes

Temperature	p	n	Entropy
Hot	2	2	1
Mild	4	2	0.918
Cool	3	1	0.811

Table 2.8.3 : Relation of attribute temperature with play cricket

Now we calculate the average information entropy for Temperature

$$I(\text{Temperature}) = \frac{p(\text{hot}) + n(\text{hot})}{p + n} * \text{Entropy}(\text{Temperature} = \text{hot}) + \frac{p(\text{mild}) + n(\text{mild})}{p + n} *$$

$$\text{Entropy}(\text{Temperature} = \text{mild}) + \frac{p(\text{cool}) + n(\text{cool})}{p + n} * \text{Entropy}(\text{Temperature} = \text{cool})$$

$$\text{i.e. } I(\text{Temperature}) = \frac{2+2}{9+5} * 1 + \frac{4+2}{9+5} * 0.918 + \frac{3+1}{9+5} * 0.811 = \mathbf{0.911}$$

Now we calculate information gain for the attribute temperature

$$\text{Gain} = \text{Entropy}(S) - I(\text{Temperature}) = 0.940 - 0.911 = \mathbf{0.029}$$

Now, repeat the same procedure for finding the entropy for Humidity as shown in Table 2.8.4.

Humidity	p	n	Entropy
High	3	4	0.985
Normal	6	1	0.591

Table 2.8.4 : Entropy of humidity

$$I(\text{Humidity}) = \frac{3+4}{9+5} * 0.985 + \frac{6+1}{9+5} * 0.591 = 0.788$$

Now we calculate gain for the attribute Humidity

$$\text{Gain} = \text{Entropy}(S) - I(\text{Humidity}) = 0.940 - 0.788 = 0.152$$

Now, repeat the same procedure for finding the entropy for Windy as shown in Table 2.8.5.

Windy	p	n	Entropy
Strong	3	3	1
Weak	6	2	0.811

Table 2.8.5 : Entropy of windy

$$I(\text{Windy}) = \frac{3+3}{9+5} * 1 + \frac{6+2}{9+5} * 0.811 = 0.892$$

Now we calculate gain for the attribute Windy

$$\text{Gain} = \text{Entropy}(S) - I(\text{Windy}) = 0.940 - 0.892 = 0.048$$

Finally the Attributes and their information gain is shown in Table IX. From this table we conclude that the outlook has maximum gain than others. So, root node for decision tree will be selected as outlook which is shown in Table 2.8.6.

Attribute	Information Gain
Outlook	0.247
Temperature	0.029
Humidity	0.152
Windy	0.048

← Selected as a Root

Table 2.8.6 : Information Gain for Different Attributes

So, the initial decision tree will look like Fig. 2.8.1.

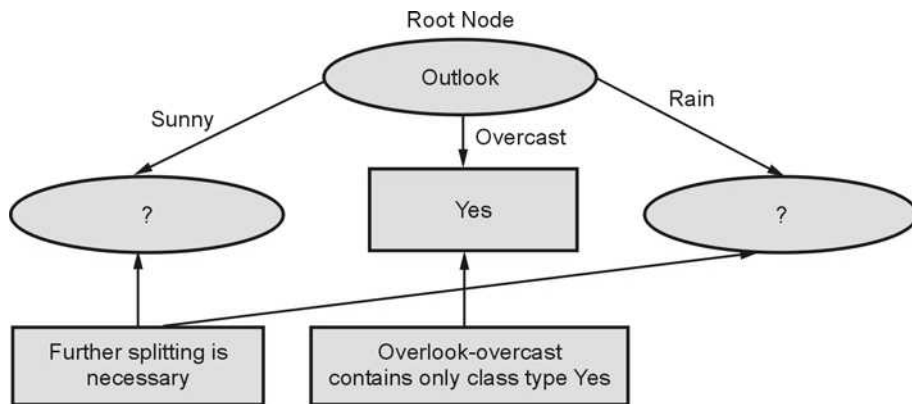


Fig. 2.8.1 : Initial decision tree with root node outlook

As seen for overcast, there is only outcome “Yes” because of which further splitting is not required and “Yes” becomes the leaf node. Whereas the sunny and rain has to be further splitted. So a new data set is created and the process is again repeated. Now consider the new tables for Outlook=Sunny and Outlook=Rainy as shown in Table 2.8.7.

Outlook	Temperature	Humidity	Windy	Play Cricket	Outlook = Sunny
Sunny	Hot	High	Weak	No	
Sunny	Hot	High	Strong	No	
Sunny	Mild	High	Weak	No	
Sunny	Cool	Normal	Weak	Yes	
Sunny	Mild	Normal	Strong	Yes	

Outlook	Temperature	Humidity	Windy	Play Cricket	Outlook = Rainy
Rainy	Mild	High	Weak	Yes	
Rainy	Cool	Normal	Weak	Yes	
Rainy	Cool	Normal	Strong	No	
Rainy	Mild	Normal	Weak	Yes	
Rainy	Mild	High	Strong	No	

Table 2.8.7 : Attribute outlook with sunny and rainy

Now we solve for attribute Outlook=Sunny. As seen from the Table 2.8.8, for Outlook= Sunny the play cricket has p=2 and n=3.

Outlook	Temperature	Humidity	Windy	Play Cricket	P = 2 N = 3 Total = 5
Sunny	Hot	High	Weak	No	
Sunny	Hot	High	Strong	No	
Sunny	Mild	High	Weak	No	
Sunny	Cool	Normal	Weak	Yes	
Sunny	Mild	Normal	Strong	Yes	

Table 2.8.8 : Outlook=Sunny

Calculate the dataset entropy for the above table

$$\text{Entropy} = \frac{-2}{2+3} * \log_2\left(\frac{2}{2+3}\right) - \frac{3}{2+3} * \log_2\left(\frac{3}{2+3}\right) = 0.971$$

We calculate the information gain for Humidity and as seen from the Table 2.8.9, the entropy for Humidity with outcome high and normal is 0.

Outlook	Humidity	Play Cricket
Sunny	High	No
Sunny	High	No
Sunny	High	No
Sunny	Normal	Yes
Sunny	Normal	Yes

Humidity	p	n	Entropy
High	0	3	0
Normal	2	0	0

Table 2.8.9 : Entropy for humidity

As I (Humidity) =0, hence information gain=0.971

Similarly, we find the Entropy and information gain for Windy as shown in Table 2.8.10.

Outlook	Windy	Play Cricket
Sunny	Strong	No
Sunny	Strong	Yes
Sunny	Weak	No
Sunny	Weak	No
Sunny	Weak	Yes

Windy	p	n	Entropy
Strong	1	1	1
Weak	1	2	0.918

Table 2.8.10 : Entropy for windy

As I (Windy) =0.951, hence information gain=0.971-0.951=0.020

Similarly, we find the Entropy and information gain for temperature as shown in Table 2.8.11.

Outlook	Temperature	Play Cricket
Sunny	Cool	Yes
Sunny	Hot	No
Sunny	Hot	No
Sunny	Mild	No
Sunny	Mild	Yes

Temperature	p	n	Entropy
Cool	1	0	0
Hot	0	2	0
Mild	1	1	1

Table 2.8.11 : Entropy for temperature

As $I(\text{Temperature}) = 0.951$, hence information gain $= 0.971 - 0.4 = 0.571$

The Information gain for the attributes is shown in Table 2.8.12.

Attribute	Information Gain
Temperature	0.571
Humidity	0.971
Windy	0.02

Selected as a Next

Table 2.8.12 : Information gain for attributes

From the Table 2.8.12, it is seen that the Humidity has the highest gain amongst the other attributes. So, it will be selected as a next node as shown in Fig. 2.8.2.

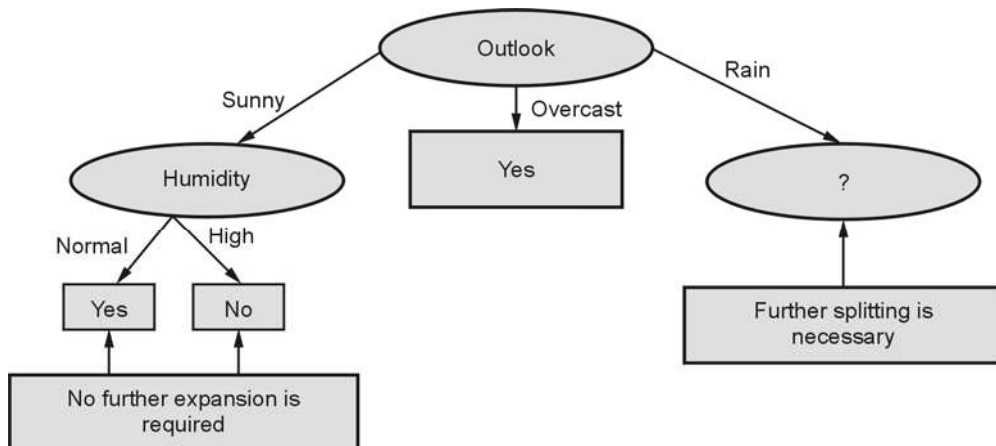


Fig. 2.8.2 : Decision tree with selected node humidity

As seen from Figure 2.5, for Humidity there are only two conditions as Normal “Yes” and High “No”. So, further expansion is not required and both become the leaf node. Now consider the new tables for Outlook=Rainy as shown in Table 2.8.13.

Outlook	Temperature	Humidity	Windy	Play Cricket	P = 3 N = 2 Total = 5
Rainy	Mild	High	Weak	Yes	
Rainy	Cool	Normal	Weak	Yes	
Rainy	Cool	Normal	Strong	No	
Rainy	Mild	Normal	Weak	Yes	
Rainy	Mild	Normal	Strong	No	

Table 2.8.13 : Attribute outlook with rainy

Now, calculate the dataset entropy for the above table.

Entropy = 0.971

For each attribute like Humidity, calculate the entropy for High and Normal as shown in Table 2.8.14.

Outlook	Humidity	Play Cricket
Rainy	High	Yes
Rainy	High	No
Rainy	Normal	Yes
Rainy	Normal	No
Rainy	Normal	Yes

Attribute	Entropy
High	1
Normal	0.918

Table 2.8.14 : Entropy for attribute humidity (High and normal)

Therefore, $I(\text{Humidity}) = 0.951$ and $\text{Information Gain} = 0.971 - 0.951 = 0.020$.

For the attribute Windy, calculate the entropy for Strong and Weak as shown in Table 2.8.15.

Outlook	Windy	Play Cricket
Rainy	Strong	No
Rainy	Strong	No
Rainy	Weak	Yes
Rainy	Weak	Yes
Rainy	Weak	Yes

Attribute	p	n	Entropy
Strong	0	2	0
Weak	3	0	0

Table 2.8.15 : Entropy for attribute windy (Strong and weak)

Therefore, $I(\text{Windy}) = 0$ and $\text{Information Gain} = 0.971$.

For each attribute like Temperature calculate the entropy for Cool, and Mild as shown in Table 2.8.16.

Outlook	Temperature	Play Cricket
Rainy	Mild	Yes
Rainy	Cool	Yes
Rainy	Cool	No
Rainy	Mild	Yes
Rainy	Mild	No

Attribute	p	n	Entropy
Cool	1	1	1
Mild	2	1	0.918

Table 2.8.16 : Entropy for attribute cool and mild

Therefore, Entropy for temperature, $I(\text{Temperature}) = 0.951$ and Information Gain=0.020.

The following Table 2.8.17 shows the information gain for all the attributes. From this table it is observed, the windy has highest gain so it will be selected as next node.

Attribute	Information Gain
Humidity	0.02
Windy	0.971
Temperature	0.02

Table 2.8.17 : Information gain for attributes

So, accordingly the decision tree is formed as shown in Fig. 2.8.3.

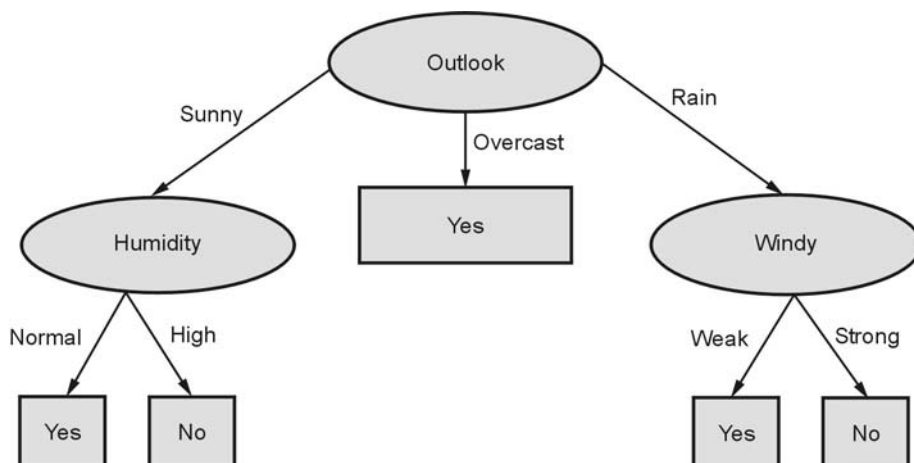


Fig. 2.8.3 : Final decision tree

As seen from Fig. 2.8.3, for Windy there are only two conditions as Weak “Yes” and Strong “No”. So, further expansion is not required and both become the leaf node. So, this becomes the final decision tree. Hence, given the attributes and decisions we can easily construct the decision tree using ID3 algorithm.

2.8.2 CART

Classification and Regression Tree (CART) is one of commonly used Decision Tree algorithm. It uses recursive partitioning approach where each of the input node is split into two child nodes. Therefore, CART decision tree is often called Binary Decision Tree. In CART, at each level of decision tree, the algorithm identify a condition - which variable and level to be used for splitting the input node (data sample) into two child nodes and accordingly build the decision tree.

CART is an alternate decision tree algorithm which can handle both regression and classification tasks. This algorithm uses a new metric named gini index to create decision points for classification tasks. Given the attributes and the decision as shown in Table 2.8.18. The procedure for creating the decision tree using CART is explained below.

Day	Outlook	Temperature	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Table 2.8.18 : Attributes/Features for CART problem

The Gini index is a measure for classification tasks in CART algorithm which has sum of squared probabilities of each class. It is defined as

GiniIndex (Attribute = value) = GI (v) = $1 - \sum (P_i)^2$ for $i = 1$ to number of classes.

Gini Index (Attribute) = $\sum_{v = \text{values}} P_v * GI(v)$

Outlook is attribute which can be either sunny, overcast or rain. Summarizing the final decisions for outlook feature is given in Table 2.8.19.

Outlook	Yes	No	Number of instances
Sunny	2	3	5
Overcast	4	0	4
Rain	3	2	5

Table 2.8.19 : Decision table for outlook attribute

Using the above information from the table, we calculate Gini(Outlook) by using the formulae's defined earlier

$$\text{Gini(Outlook = Sunny)} = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 1 - 0.16 - 0.36 = 0.48$$

$$\text{Gini(Outlook=Overcast)} = 1 - \left(\frac{4}{4}\right)^2 - \left(\frac{0}{4}\right)^2 = 0$$

$$\text{Gini(Outlook=Rain)} = 1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 1 - 0.36 - 0.16$$

Then, we will calculate weighted sum of gini indexes for outlook feature.

$$\text{Gini(Outlook)} = \left(\frac{5}{14}\right) \times 0.48 + \left(\frac{4}{14}\right) \times 0 + \left(\frac{5}{14}\right) \times 0.48 = 0.171 + 0 + 0.171 = 0.342$$

Here after, the same procedure is repeated for other attributes. Temperature is an attribute which has 3 different values : Cool, Hot and Mild. The summary of decisions for temperature is given in Table 2.8.20.

Temperature	Yes	No	Number of instances
Hot	2	2	4
Cool	3	1	4
Mild	4	2	6

Table 2.8.20 : Decision table for temperature attribute

$$\text{Gini}(\text{Temp} = \text{Hot}) = 1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2 = 0.5$$

$$\text{Gini}(\text{Temp} = \text{Cool}) = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 1 - 0.5625 - 0.0625 = 0.375$$

$$\text{Gini}(\text{Temp} = \text{Mild}) = 1 - \left(\frac{4}{6}\right)^2 - \left(\frac{2}{6}\right)^2 = 1 - 0.444 - 0.111 = 0.445$$

We'll calculate weighted sum of gini index for temperature feature

$$\text{Gini}(\text{Temp}) = \left(\frac{4}{14}\right) \times 0.5 + \left(\frac{4}{14}\right) \times 0.375 + \left(\frac{6}{14}\right) \times 0.445 = 0.142 + 0.107 + 0.190 = 0.439$$

Humidity is a binary class feature. It can be high or normal as shown in Table 2.8.21.

Humidity	Yes	No	Number of instances
High	3	4	7
Normal	6	1	7

Table 2.8.21 : Decision table for humidity attribute

$$\text{Gini}(\text{Humidity} = \text{High}) = 1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2 = 1 - 0.183 - 0.326 = 0.489$$

$$\text{Gini}(\text{Humidity} = \text{Normal}) = 1 - \left(\frac{6}{7}\right)^2 - \left(\frac{1}{7}\right)^2 = 1 - 0.734 - 0.02 = 0.244$$

Weighted sum for humidity feature will be calculated next

$$\text{Gini}(\text{Humidity}) = \left(\frac{7}{14}\right) \times 0.489 + \left(\frac{7}{14}\right) \times 0.244 = 0.367$$

Wind is a binary class similar to humidity. It can be weak and strong as shown in Table 2.8.22.

Wind	Yes	No	Number of instances
Weak	6	2	8
Strong	3	3	6

Table 2.8.22 : Decision table for wind attribute

$$\text{Gini}(\text{Wind} = \text{Weak}) = 1 - \left(\frac{6}{8}\right)^2 - \left(\frac{2}{8}\right)^2 = 1 - 0.5625 - 0.062 = 0.375$$

$$\text{Gini}(\text{Wind} = \text{Strong}) = 1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 = 1 - 0.25 - 0.25 = 0.5$$

$$\text{Gini}(\text{Wind}) = \left(\frac{8}{14}\right) \times 0.375 + \left(\frac{6}{14}\right) \times 0.5 = 0.428$$

After calculating the gini index for each attribute, the attribute having minimum value is selected as the node. So, from the Table 2.8.23 the outlook feature has the minimum value, therefore, outlook attribute will be at the top of the tree as shown in Fig. 2.8.4.

Feature	Gini index
Outlook	0.342
Temperature	0.439
Humidity	0.367
Wind	0.428

Table 2.8.23 : Gini index for each feature

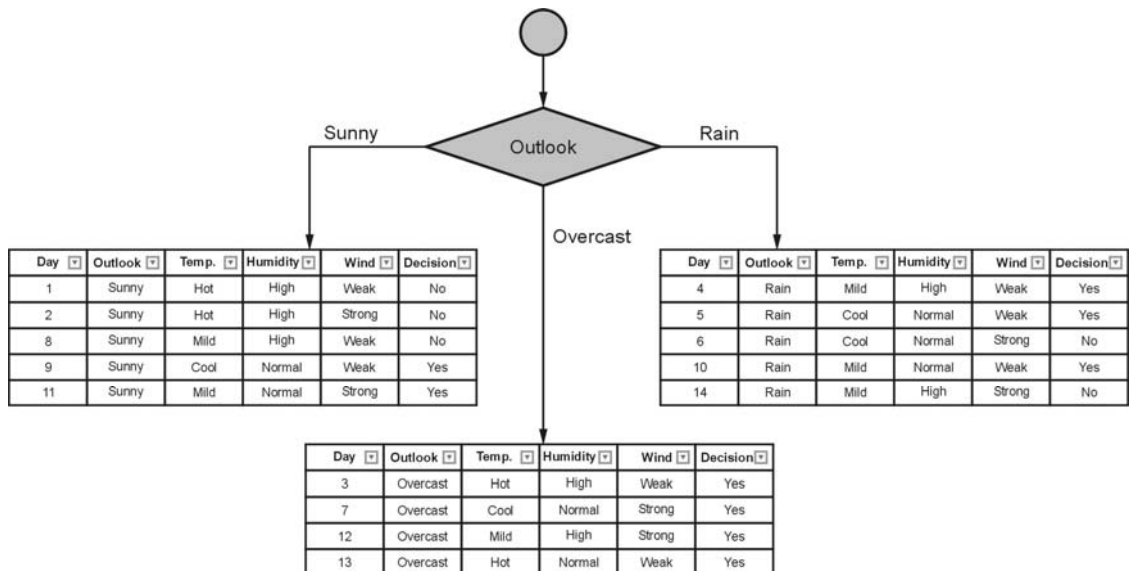


Fig. 2.8.4 : Representation of outlook as a top node

From Fig. 2.8.4, we can see that the sub dataset Overcast has only yes decisions, which means the leaf node for Overcast is “Yes” as shown in Fig. 2.8.5 which will not require further expansion.

We will apply same principles to the other sub datasets in the following steps. Let us take sub dataset for Outlook=Sunny. We need to find the gini index scores for temperature, humidity and wind features respectively. The sub dataset for Outlook=Sunny is as shown in Table 2.8.24.

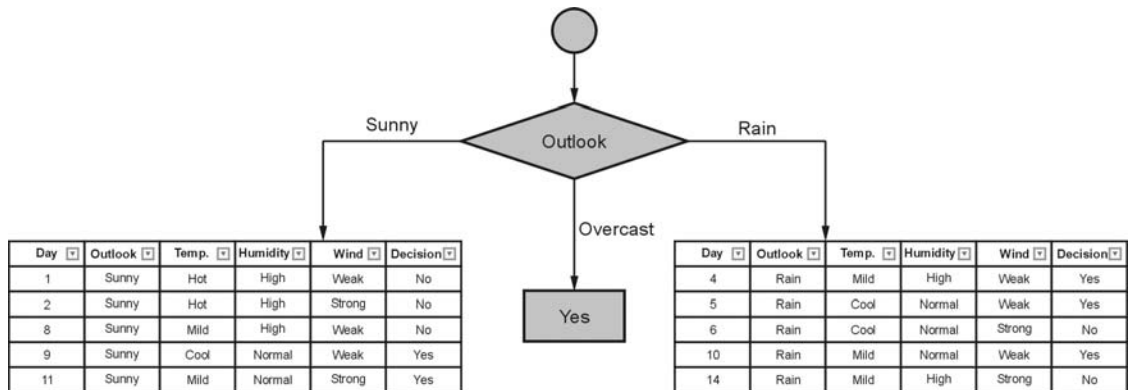


Fig. 2.8.5 : Representation of leaf node for overcast

Day	Outlook	Temperature	Humidity	Wind	Decision
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes

Table 2.8.24 : Sub dataset of outlook=sunny

Now, we determine Gini of temperature for Outlook=Sunny as per Table 2.8.25.

Temperature	Yes	No	Number of instances
Hot	0	2	2
Cool	1	0	1
Mild	1	1	2

Table 2.8.25 : Decision table of temperature for outlook=sunny

$$\text{Gini}(\text{Outlook} = \text{Sunny and Temperature} = \text{Hot}) = 1 - \left(\frac{0}{2}\right)^2 - \left(\frac{2}{2}\right)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Temperature} = \text{Cool}) = 1 - \left(\frac{1}{1}\right)^2 - \left(\frac{0}{1}\right)^2 = 0$$

$$\text{Gini}(\text{Outlook} = \text{Sunny and Temperature} = \text{Mild}) = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 1 - 0.25 - 0.25 = 0.5$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Temperature}) = \left(\frac{2}{5}\right) \times 0 + \left(\frac{1}{5}\right) \times 0 + \left(\frac{2}{5}\right) \times 0.5 = 0.2$$

Now, we determine Gini of humidity for Outlook=Sunny as per Table 2.8.26.

Humidity	Yes	No	Number of instances
High	0	3	3
Normal	2	0	2

Table 2.8.26 : Decision table of humidity for outlook=sunny

$$\text{Gini}(\text{Outlook}=\text{Sunny and Humidity}=\text{High}) = 1 - \left(\frac{0}{3}\right)^2 - \left(\frac{3}{3}\right)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Humidity}=\text{Normal}) = 1 - \left(\frac{2}{2}\right)^2 - \left(\frac{0}{2}\right)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Humidity}) = \left(\frac{3}{5}\right) \times 0 + \left(\frac{2}{5}\right) \times 0 = 0$$

Now, we determine Gini of Wind for Outlook=Sunny as per Table 2.8.27.

Wind	Yes	No	Number of instances
Weak	1	2	3
Strong	1	1	2

Table 2.8.27 : Decision table of wind for outlook=sunny

$$\text{Gini}(\text{Outlook}=\text{Sunny and Wind}=\text{Weak}) = 1 - \left(\frac{1}{3}\right)^2 - \left(\frac{2}{3}\right)^2 = 0.266$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Wind}=\text{Strong}) = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 0.2$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Wind}) = \left(\frac{3}{5}\right) \times 0.266 + \left(\frac{2}{5}\right) \times 0.2 = 0.466$$

Decision for sunny outlook

We've calculated gini index scores for features when outlook is sunny as shown in Table 2.8.28. The winner is humidity because it has the lowest value.

Feature	Gini index
Temperature	0.2

Humidity	0
Wind	0.466

Table 2.8.28 : Gini index of each feature for outlook=sunny

Humidity is the extension of Outlook Sunny as the gini index for Humidity is minimum as shown in Fig. 2.8.6.

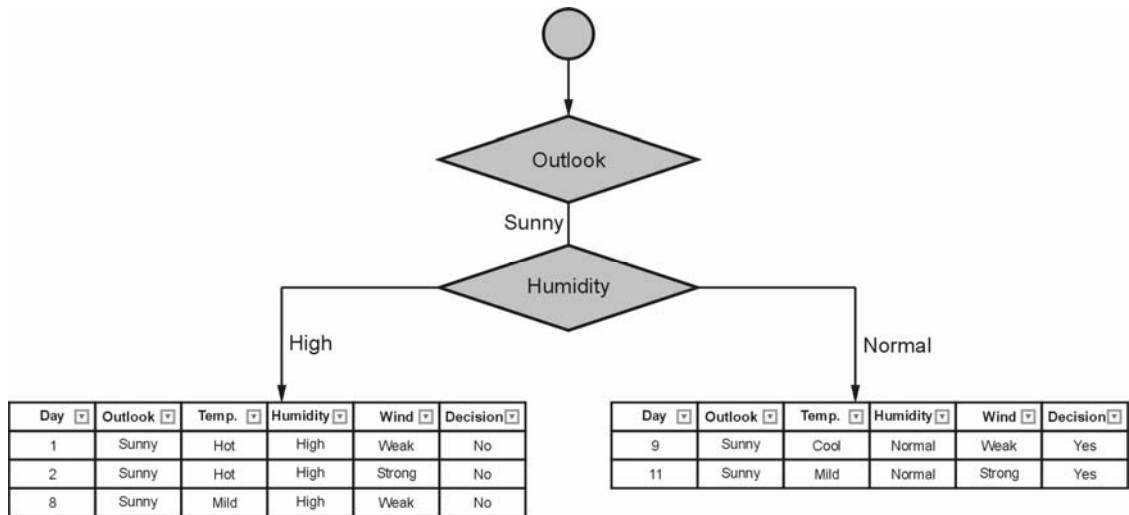


Fig. 2.8.6 : Node of humidity for outlook=sunny

As seen from Fig. 2.8.6, decision is always no for high humidity and sunny outlook. On the other hand, decision will always be yes for normal humidity and sunny outlook. Therefore, this branch is over and the leaf nodes of Humidity for Outlook=Sunny is shown in Fig. 2.8.7.

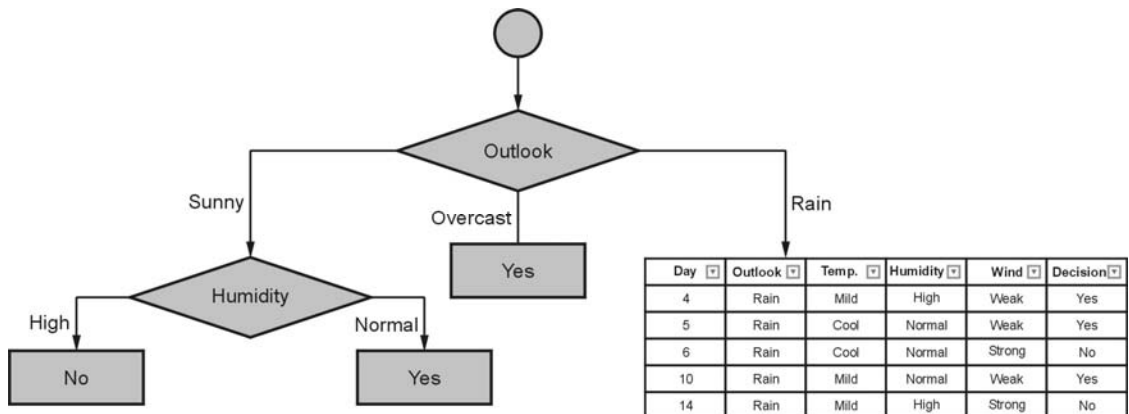


Fig. 2.8.7 : Leaf node of humidity for outlook=sunny

Let us take sub dataset for Outlook= Rain, and determine the gini index for temperature, humidity and wind features respectively. The sub dataset for Outlook= Rain is as shown in Table 2.8.29.

Day	Outlook	Temperature	Humidity	Wind	Decision
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
10	Rain	Mild	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Table 2.8.29 : Sub dataset of outlook=rain

The calculation for gini index scores for temperature, humidity and wind features when outlook is rain is shown following Tables 2.8.30, 2.8.31, and 2.8.32 .

Temperature	Yes	No	Number of instances
Cool	1	1	2
Mild	2	1	3

Table 2.8.30 : Decision table of temperature for outlook=rain

$$\text{Gini(Outlook=Rain and Temperature=Cool)} = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 0.5$$

$$\text{Gini(Outlook=Rain and Temperature=Mild)} = 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 0.444$$

$$\text{Gini(Outlook=Rain and Temperature)} = \left(\frac{2}{5}\right) \times 0.5 + \left(\frac{3}{5}\right) \times 0.444 = 0.466$$

Table 2.8.31 : Decision table of humidity for outlook=rain

Humidity	Yes	No	Number of instances
High	1	1	2
Normal	2	1	3

$$\text{Gini(Outlook=Rain and Humidity=High)} = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 0.5$$

$$\text{Gini(Outlook=Rain and Humidity=Normal)} = 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 0.444$$

$$\text{Gini(Outlook=Rain and Humidity)} = \left(\frac{2}{5}\right) \times 0.5 + \left(\frac{3}{5}\right) \times 0.444 = 0.466$$

Wind	Yes	No	Number of instances
Weak	3	0	3
Strong	0	2	2

Table 2.8.32 : Decision table of wind for outlook=rain

$$\text{Gini}(\text{Outlook}=\text{Rain and Wind}=\text{Weak}) = 1 - \left(\frac{3}{3}\right)^2 - \left(\frac{0}{3}\right)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Wind}=\text{Strong}) = 1 - \left(\frac{0}{2}\right)^2 - \left(\frac{2}{2}\right)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Wind}) = \left(\frac{3}{5}\right) \times 0 + \left(\frac{2}{5}\right) \times 0 = 0$$

The winner is wind feature for rain outlook because it has the minimum gini index score in features as per Table 2.8.33.

Feature	Gini index
Temperature	0.466
Humidity	0.466
Wind	0

Table 2.8.33 : Gini index of each feature for Outlook=Rain

Place the wind attribute for outlook rain branch and monitor the new sub data sets as shown in Figure 2.8.8.

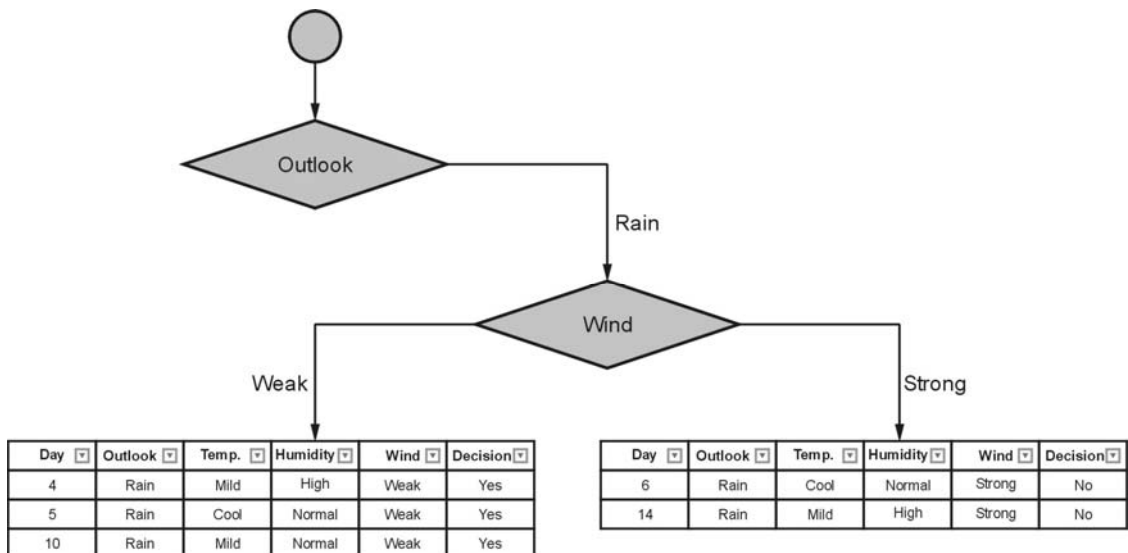


Fig. 2.8.8 : Node of Wind for Outlook=Rain

As seen, when wind is weak the decision is always yes. On the other hand, if wind is strong the decision is always no. This means that this branch is over and the final decision tree using CART algorithm is depicted in Fig. 2.8.9.

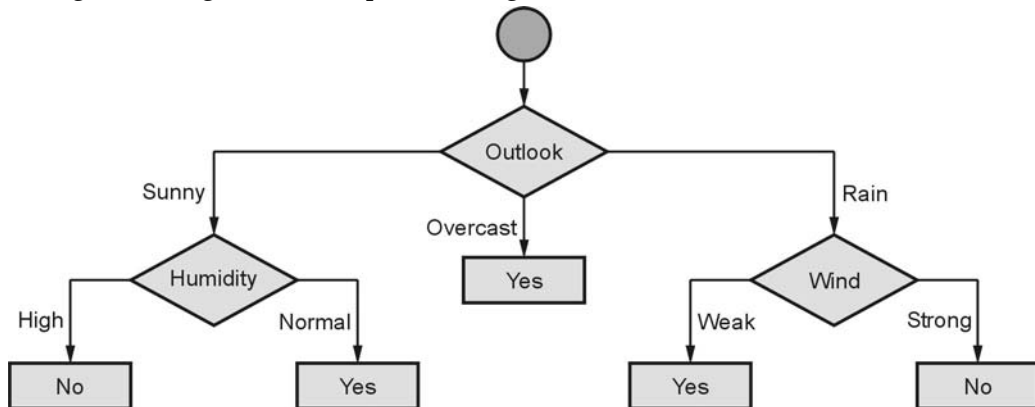


Fig. 2.8.9 : Final decision tree using CART algorithm

2.8.3 C4.5

The C4.5 algorithm is used to generate a Decision tree based on Decision Tree Classifier. It is mostly used in data mining where decisions are generated based on a certain sample of data. It has many improvements over the original ID3 algorithm. The C4.5 algorithm can handle missing data

So, If the training records contains unknown attribute values then C4.5 evaluates the gain for each attribute by considering only the records where the attribute is defined. For the corresponding records of each partition, the gain is calculated, and the partition that maximizes the gain is chosen for the next split. It also supports both categorical and continuous attributes where values of a continuous variable are sorted and partitioned. The ID3 algorithm may construct a deep and complex tree, which would cause overfitting. The C4.5 algorithm addresses the overfitting problem in ID3 by using a bottom-up technique called pruning to simplify the tree by removing the least visited nodes and branches.

2.9 Evaluating Decision Tree

The decision tree often uses greedy algorithms to choose the option that seems the best available at that moment. At each step, the algorithm selects the attribute to be used for splitting the remaining records.

This selection of attribute may not be the best overall, but it is guaranteed to be the best at that step. This characteristic strengthens the effectiveness of decision trees. However, selecting the wrong attribute with bad split may propagate through the rest of the tree. Thus, to address this issue, the synergistic method can be utilized like random forest which may randomize the splitting or even randomize data and come up with numerous tree structure. These trees at that point vote in favor of each class, and the class with the most votes is picked as the predicted class.

There are few ways to evaluate a decision tree. Some of the important evaluations are given as follows.

Firstly, evaluate whether the splits of the tree make sense. Conduct stability checks by validating the decision rules with domain experts, and determine if the decision rules are sound. Second, look at the depth and nodes of the tree. Having such a large number of layers and getting nodes with few members might be signs of overfitting. In overfitting, the model fits the training set well, however it performs ineffectively on the new samples in the testing set.

For decision tree learning, overfitting can be caused by either the lack of training data or the biased data in the training set. So, to avoid overfitting in decision tree two methodologies can be utilized. First is stop rising the tree early before it reaches the point where all the training data is perfectly classified and second is grow the full tree, and then post-prune the tree with methods such as reduced-error pruning and rule-based post pruning and Lastly, use standard diagnostics tools that apply to classifiers that can help evaluate overfitting.

The structure of a decision tree is sensitive to small variations in the training data. Therefore, constructing two decision trees based on two different subsets of same dataset may result in very different trees and it is not a good choice if the dataset contains many irrelevant or redundant variables. If the dataset contains redundant variables, the resulting decision tree ignores all and algorithm cannot able to detect the information gain on other hand if dataset contains irrelevant variables and accidentally chosen as splits in the tree, the tree may grow too large and may end up with less data at every split, where overfitting is likely to occur.

Although decision trees are able to handle correlated variables, as when most of the variables in the training set are correlated, overfitting is likely to occur. To overcome the issue of instability and potential overfitting, one can combine the decisions of several randomized shallow decision trees using classifier called random forest

For binary decisions, a decision tree works better if the training dataset consists of records with an even probability of each result. In that scenario, the logistic regression on a dataset with multiple variables can be used to determine which variables are the most useful to select based on information gain.

2.10 Decision Tree in R

In R programming, the decision tree can be plotted using package called `rpart.plot`. The common steps for implementing Decision tree in R are as follows :

Step 1 : Import the data

Step 2 : Clean the dataset

Step 3 : Create train/test set

Step 4 : Build the model

Step 5 : Make prediction

Step 6 : Measure performance

Step 7 : Tune the hyper-parameters

The implementation of Decision tree in R is briefly explained in Practical section.

2.11 Baye's Theorem

Bayes' theorem is also called Bayes' Rule or Bayes' Law and is the foundation of the field of Bayesian statistics. Bayes' Theorem was named after 18th century mathematician Thomas Bayes. Bayes' Theorem allows you to update predicted probabilities of an event by incorporating new information. It is often employed in finance in updating risk evaluation.

The probability of two events A and B happening, i.e. $P(A \cap B)$, is the probability of A, i.e. $P(A)$, times the probability of B given that A has occurred, $P(B|A)$.

$$P(A \cap B) = P(A)P(B|A)$$

Similarly, it is also equal to the probability of B times the probability of A given B.

$$P(A \cap B) = P(B)P(A|B)$$

Equating the two yields,

$$P(B)P(A|B) = P(A)P(B|A)$$

$$P(A|B) = P(A) P(B|A) P(B)$$

This equation is known as Bayes Theorem, which relates the conditional and marginal probabilities of stochastic events A and B as

$$P(A|B) = P(B|A) P(A) / P(B) .$$

Each term in Bayes' theorem has a conventional name. $P(A)$ is the prior probability or marginal probability of A. It is "prior" in the sense that it does not take into account any information about B. $P(A|B)$ is the conditional probability of A, given B. It is also called the posterior probability because it is derived from or depends upon the specified value of B. $P(B|A)$ is the conditional probability of B given A. $P(B)$ is the prior or marginal probability of B, and acts as a normalizing constant. This theorem plays an important role in determining the probability of the event, provided the prior knowledge of another event.

2.12 Naive Bayes Classifier

It is a classification technique based on Bayes' Theorem with an assumption of independence among predictors. In simple terms, a Naive Bayes classifier assumes that the presence of a particular feature in a class is unrelated to the presence of any other feature. Even if these features depend on each other or upon the existence of the other features, all of these properties independently contribute to the probability. Naive Bayes model is easy to build and particularly useful for very large data sets. Along with simplicity, Naive Bayes is known to outperform even highly sophisticated classification methods. Naive Bayes is a probabilistic classifier inspired by the Bayes theorem. Under a simple assumption which is the attributes are conditionally independent.

$$P(c | x) = \frac{P(x | c) P(c)}{P(x)}$$

Likelihood Class prior probability
 ↓ ↓
 Posterior probability Predictor prior probability

$$P(c | x) = P(x_1 | c) \times P(x_2 | c) \times \dots \times P(x_n | c) \times P(c)$$

The classification is conducted by deriving the maximum posterior which is the maximal $P(c|X)$ with the above assumption applying to Bayes theorem. This assumption greatly reduces the computational cost by only counting the class distribution.

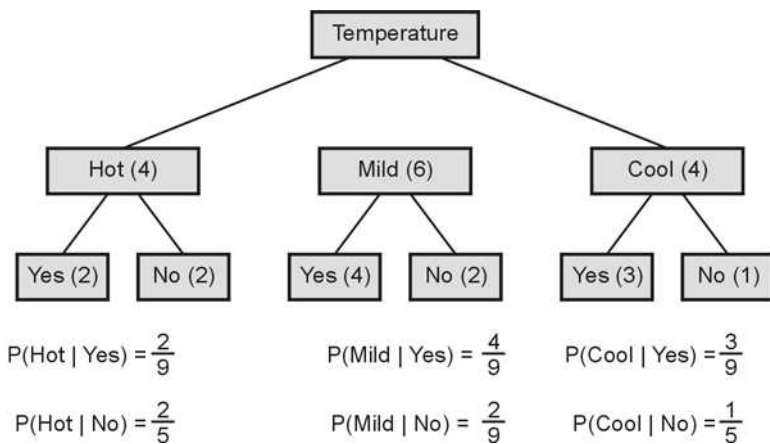
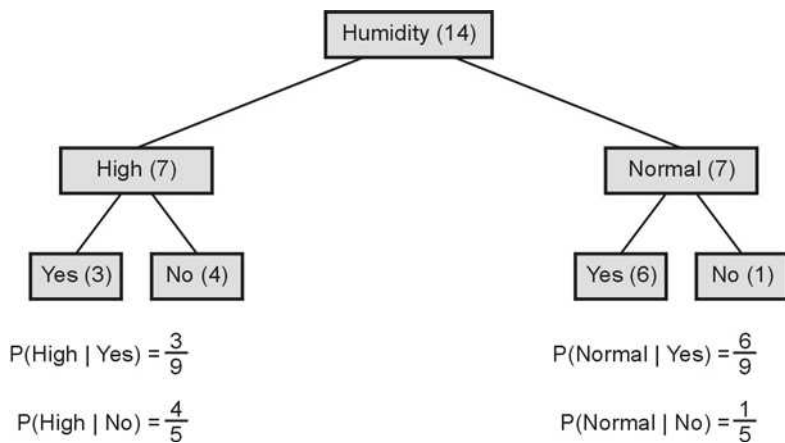
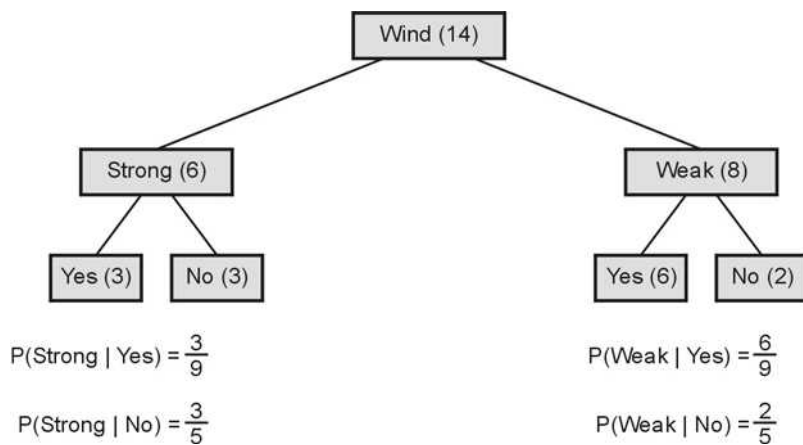
Given the data set with classification and then using the condition which is not present in the data set we can determine the appropriate class. Consider the Table 2.12.1, find the probability of playing Cricket with outcome Yes or No when conditions are

Temperature = Cool, Humidity = High, Wind = Strong and Outlook = Sunny which is not in the Table 2.12.1.

Day	Outlook	Temperature	Humidity	Wind	Play cricket
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	High	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Table 2.12.1 : Attributes for Naive Bayes classifier problem

From the Table 2.12.1, it is seen the attribute play cricket has outcome Yes = 9 and No = 5 for 14 conditions. Using the table, we determined $P(\text{Strong} \mid \text{Yes})$, $P(\text{Strong} \mid \text{No})$, $P(\text{Weak} \mid \text{Yes})$, $P(\text{Weak} \mid \text{No})$, $P(\text{High} \mid \text{Yes})$, $P(\text{High} \mid \text{No})$, $P(\text{Normal} \mid \text{Yes})$, $P(\text{Normal} \mid \text{No})$, $P(\text{Hot} \mid \text{Yes})$, $P(\text{Hot} \mid \text{No})$, $P(\text{Mild} \mid \text{Yes})$, $P(\text{Mild} \mid \text{No})$, $P(\text{Cool} \mid \text{Yes})$, $P(\text{Cool} \mid \text{No})$, $P(\text{Sunny} \mid \text{Yes})$, $P(\text{Sunny} \mid \text{No})$, $P(\text{Overcast} \mid \text{Yes})$, $P(\text{Overcast} \mid \text{No})$ and $P(\text{Rain} \mid \text{Yes})$, $P(\text{Rain} \mid \text{No})$ as shown in Fig. 2.12.1.



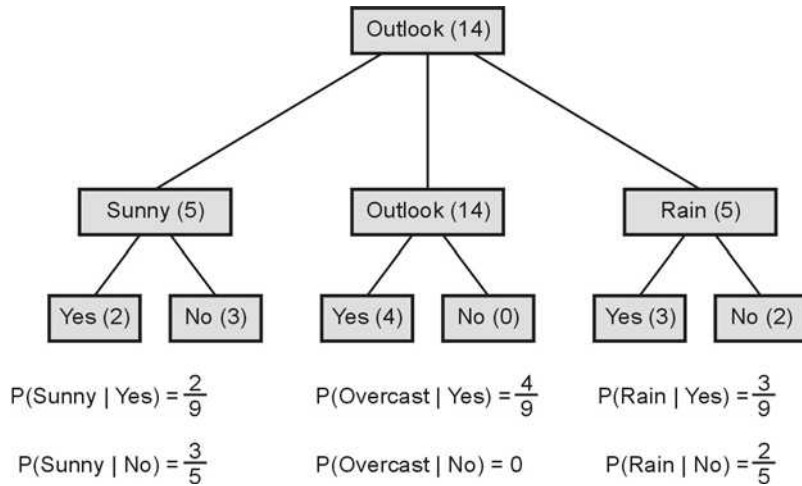


Fig. 2.12.1 : Conditional probabilities for different attributes

Consider the first attribute Wind, in which we have two possibilities like Strong and Weak. From the table, it is seen that Probability of Wind i.e. $P(\text{Wind}) | \text{Yes}$ Finding the probability of wind is demonstrated in Fig. 2.12.1.

As seen from the Fig. 2.12.1, the Wind has two subparameters like Strong and Weak. From Table 2.12.2, number of Strong are appeared 6 times while Weak appeared at 8 times. Now number of Yes for Strong are 3 and Number of No for Strong are 3 therefor we can say $P(\text{Strong} | \text{Yes}) = \frac{3}{9}$ and $P(\text{Strong} | \text{No}) = \frac{3}{5}$. Similarly, for Weak, Number of yes are 6 and No's are 2. So $P(\text{Weak} | \text{Yes}) = \frac{6}{9}$ and $P(\text{Weak} | \text{No}) = \frac{2}{5}$. Similarly for Humidity, two conditions are High and Normal. From Table, number of High's are 7 while Normal appeared 7 times. Now number of Yes for High are 3 and No are 4. therefor we can say $P(\text{High} | \text{Yes}) = \frac{3}{9}$ and $P(\text{High} | \text{No}) = \frac{4}{5}$. Similarly, for Normal, Number of yes are 6 and No's are 1. So $P(\text{Normal} | \text{Yes}) = \frac{6}{9}$ and $P(\text{Normal} | \text{No}) = \frac{1}{5}$. Similarly for Temperature and Outlook, the probabilities are given as shown in Table 2.12.2.

Temperature :		
$P(\text{Hot} \text{Yes}) = \frac{2}{9}$	$p(\text{Mild} \text{Yes}) = \frac{4}{9}$	$p(\text{Cool} \text{Yes}) = \frac{3}{9}$
$P(\text{Hot} \text{No}) = \frac{2}{5}$	$p(\text{Mild} \text{No}) = \frac{2}{9}$	$p(\text{Cool} \text{No}) = \frac{1}{5}$

Outlook :

$$P(\text{Sunny} \mid \text{Yes}) = \frac{2}{9}$$

$$p(\text{Overcast} \mid \text{Yes}) = \frac{4}{9}$$

$$p(\text{Rain} \mid \text{Yes}) = \frac{3}{9}$$

$$P(\text{Sunny} \mid \text{No}) = \frac{3}{5}$$

$$p(\text{Overcast} \mid \text{No}) = 0$$

$$p(\text{Rain} \mid \text{No}) = \frac{2}{5}$$

Table 2.12.2 : Conditional probabilities for attributes temperature and outlook

Now Considering the problem statement, Let $X = \{\text{Sunny, Cool, High, Strong}\}$ and using the relation we can write

$$P(X \mid \text{Yes}) = P(\text{Yes}) * P(\text{Sunny} \mid \text{Yes}) * P(\text{Cool} \mid \text{Yes}) * P(\text{High} \mid \text{Yes}) * P(\text{Strong} \mid \text{Yes})$$

$$= \frac{9}{14} * \frac{2}{9} * \frac{3}{9} * \frac{3}{9} * \frac{3}{9} = 0.0053$$

$$P(X \mid \text{No}) = P(\text{No}) * P(\text{Sunny} \mid \text{No}) * P(\text{Cool} \mid \text{No}) * P(\text{High} \mid \text{No}) * P(\text{Strong} \mid \text{No})$$

$$= \frac{5}{14} * \frac{3}{5} * \frac{1}{5} * \frac{4}{5} * \frac{3}{5} = 0.0206$$

As $P(X \mid \text{No})$ is greater than $P(X \mid \text{Yes})$, the answer is No for playing cricket under these conditions i.e. Outlook is Sunny, Temperature is Cool, Humidity is High and Wind is Strong. With this approach, we can determine the answer for playing cricket as Yes or No for other conditions which are not mentioned in the Table. 2.12.2 In this way, we have learned the K-means clustering along with use cases and two statistical classifiers in this chapter.